

-  GUÍA DE INTEGRACIÓN — ELIANA
 -  ARCHIVOS QUE SON TU RESPONSABILIDAD
 -  PRE-REQUISITOS
 -  PASO A PASO COMPLETO
 - PASO 1 — Clonar el repositorio (solo si no lo tienes)
 - PASO 2 — Configurar identidad Git
 - PASO 3 — Esperar y actualizar develop
 - PASO 4 — Crear tu rama
 - PASO 5 — Verificar compilación
 - PASO 6 — Agregar SOLO tus carpetas al staging
 - PASO 7 — Revisar los cambios importantes
 - PASO 8 — Hacer commit
 - PASO 9 — Subir tu rama a GitHub
 - PASO 10 — Verificar en GitHub
 - PASO 11 — Crear el Pull Request
 -  ACTUALIZAR TU RAMA CON DEVELOP
 -  PROBLEMAS COMUNES Y SOLUCIONES
 - frontend-v2: "npm run build" falla con errores TypeScript
 - frontend-v2: node_modules apareció en git status
 - inscrip-service: error de compilación JPA
 - "Port 8084 already in use"
 - El frontend no conecta con el backend (CORS)
 -  CHECKLIST PERSONAL — ELIANA
 - Preparación
 - Verificación de compilación
 - Antes del commit
 - PR
 -  INFORMACIÓN TÉCNICA



GUÍA DE INTEGRACIÓN — ELIANA

Componentes a cargo: `inscrip-service/` + `frontend-v2/` **Rama de trabajo:** `feature/eliana-inscrip-frontend` **Ejecutar en:** FASE 5 (la última, después de que todos hayan mergeado)

ARCHIVOS QUE SON TU RESPONSABILIDAD

```
inscrip-service/
├── src/
│   ├── main/
│   │   ├── java/com/eventos/inscrip_service/
│   │   │   ├── InscripServiceApplication.java
│   │   │   ├── controller/
│   │   │   │   ├── InscriptionController.java
│   │   │   │   └── HealthController.java
│   │   │   ├── dto/
│   │   │   │   ├── InscriptionRequest.java
│   │   │   │   └── InscriptionResponse.java
│   │   │   ├── entity/
│   │   │   │   └── Inscription.java
│   │   │   ├── exception/
│   │   │   │   ├── GlobalExceptionHandler.java
│   │   │   │   └── InscriptionException.java
│   │   │   ├── mapper/
│   │   │   │   └── InscriptionMapper.java
│   │   │   ├── repository/
│   │   │   │   └── InscriptionRepository.java
│   │   │   └── service/
│   │   │       └── InscriptionService.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
├── .mvn/
├── pom.xml
└── mvnw / mvnw.cmd
```

```
frontend-v2/
├── src/
│   ├── App.tsx
│   ├── main.tsx
│   ├── index.css
│   ├── assets/
│   ├── components/
│   │   ├── Button.tsx
│   │   ├── EventCard.tsx
│   │   ├── MainLayout.tsx
│   │   ├── Modal.tsx
│   │   ├── PuraVidaLogo.tsx
│   │   └── ToastContext.tsx
│   ├── hooks/
│   │   ├── useAuth.tsx
│   │   └── useNotifications.ts
│   └── pages/
│       ├── HomePage.tsx
│       ├── AuthPage.tsx
│       └── EventsPage.tsx
```

```
| | | EventDetailPage.tsx
| | | MyEventsPage.tsx
| | | ProfilePage.tsx
| | | services/
| | |   api.ts
| | |   styles/
| | public/
| | package.json
| | tsconfig.json
| | vite.config.ts
| | frontend.Dockerfile      ← NUEVO
| | README.md
```

 Tu componente es el más visible para el usuario final: el **frontend** es lo que los usuarios ven, y el **inscrip-service** maneja el corazón del negocio (las inscripciones a eventos).



PRE-REQUISITOS

```
# Verificar Git
git --version

# Verificar Java 17 (para inscrip-service)
java -version
# Resultado esperado: openjdk version "17.x.x"

# Verificar Node.js (para frontend)
node --version
# Resultado esperado: v18.x.x o superior

# Verificar npm
npm --version
# Resultado esperado: 9.x.x o superior
```



PASO A PASO COMPLETO

PASO 1 — Clonar el repositorio (solo si no lo tienes)

```
git clone [URL_DEL_REPO]
cd Eventos-Microservicios
```

PASO 2 — Configurar identidad Git

```
git config user.name "Eliana"
git config user.email "tu-email@gmail.com"
```

PASO 3 — Esperar y actualizar develop

⚠️ **ESPERA** a que Rachel confirme que las Fases 1, 2, 3 y 4 están completas. Esto significa: scripts ✓, auth-service ✓, notification-service ✓, user-service ✓, gateway ✓, event-service ✓

```
git checkout develop
git pull origin develop
```

Verifica que están todos los commits anteriores:

```
git log --oneline -10
# Debes ver commits de Francisco, Rachel, Andrea y Braylin
```

PASO 4 — Crear tu rama

```
git checkout -b feature/eliana-inscrip-frontend
```

Confirma:

```
git branch
# * feature/eliana-inscrip-frontend
```

PASO 5 — Verificar compilación

```
# Verificar inscrip-service
cd inscrip-service
./mvnw package -DskipTests
cd ..

# Verificar frontend-v2
cd frontend-v2
npm install          # Instalar dependencias si no están
npm run build        # Compilar TypeScript + Vite
cd ..
```

Ambos deben completar sin errores.

PASO 6 — Agregar SOLO tus carpetas al staging

 **REGLA CRÍTICA:** Solo `inscrip-service/` y `frontend-v2/`. NADA más.

```
# Agregar inscrip-service
git add inscrip-service/

# Agregar frontend-v2
git add frontend-v2/

# VERIFICAR inmediatamente
git status
```

Salida esperada:

```
On branch feature/eliana-inscrip-frontend
Changes to be committed:
  modified:   frontend-v2/src/components/MainLayout.tsx
  modified:   frontend-v2/src/pages/AuthPage.tsx
  modified:   frontend-v2/src/pages/EventDetailPage.tsx
  modified:   frontend-v2/src/pages/MyEventsPage.tsx
  modified:   frontend-v2/src/services/api.ts
  new file:   frontend-v2/frontend.Dockerfile
  modified:   inscrip-
service/src/main/java/com/eventos/inscrip_service/controller/InscriptionController.
java
  modified:   inscrip-
service/src/main/java/com/eventos/inscrip_service/repository/InscriptionRepository.
```

```
java
    modified:   inscrip-
service/src/main/java/com/eventos/inscrip_service/service/InscriptionService.java
    modified:   inscrip-service/src/main/resources/application.properties
```

✅ Solo deben aparecer archivos de `inscrip-service/` y `frontend-v2/` ❌ Si aparece algo más, quítalo:

```
git restore --staged auth-service/
git restore --staged user-service/
git restore --staged gateway/
git restore --staged event-service/
git restore --staged notification-service/
```

⚠️ **IMPORTANTE sobre node_modules:** El archivo `.gitignore` ya excluye `node_modules/`. No deberían aparecer en `git status`. Si aparecen, avisa a Rachel.

PASO 7 — Revisar los cambios importantes

```
# Ver cambios en api.ts (conexión con backend)
git diff --staged frontend-v2/src/services/api.ts

# Ver cambios en InscriptionController
git diff --staged inscrip-
service/src/main/java/com/eventos/inscrip_service/controller/InscriptionController.
java
```

PASO 8 — Hacer commit

```
git commit -m "feat(inscrip+frontend): inscrip-service y frontend-v2 completos"
```

inscrip-service:

- CRUD de inscripciones a eventos
- `InscriptionController`, `InscriptionService`, `InscriptionRepository`
- `InscriptionMapper` para conversión DTO↔Entidad
- Manejo de excepciones (`GlobalExceptionHandler`, `InscriptionException`)
- `application.properties` → Railway MySQL (`inscrip_db`)

frontend-v2 (React 19 + TypeScript + Vite):

- Páginas: Home, Auth, Events, EventDetail, MyEvents, Profile
- Componentes: `EventCard`, `MainLayout`, `Modal`, `ToastContext`

- Hooks: `useAuth`, `useNotifications`
- `api.ts`: proxy local `/api` → Gateway:8080
- `frontend.Dockerfile` para despliegue"

PASO 9 — Subir tu rama a GitHub

Primera vez:

```
git push -u origin feature/eliana-inscrip-frontend
```

Cambios adicionales:

```
git push origin feature/eliana-inscrip-frontend
```

Force push cuando sea necesario:

```
git push origin feature/eliana-inscrip-frontend --force-with-lease
```

PASO 10 — Verificar en GitHub

1. Ve a GitHub → repositorio
2. Selecciona `feature/eliana-inscrip-frontend`
3. Confirma que solo hay cambios en `inscrip-service/` y `frontend-v2/`
4. Archivos críticos a verificar:
 - `frontend-v2/src/services/api.ts` — base URL del API
 - `frontend-v2/src/pages/MyEventsPage.tsx` — Vista de inscripciones
 - `inscrip-service/src/main/java/com/eventos/inscrip_service/controller/InscriptionController.java`

PASO 11 — Crear el Pull Request

Por GitHub UI:

1. Clic en "Compare & pull request"

2. Configura:

- **Base:** `develop` ← ¡crítico!
- **Compare:** `feature/eliana-inscrip-frontend`
- **Título:** `feat(inscrip+frontend): inscrip-service y frontend-v2 completos`
- **Descripción:**

```
## inscrip-service
- Inscripciones a eventos: crear, listar, cancelar
- Manejo de cupo disponible
- Integración con event-service para validación de capacidad

## frontend-v2
- SPA con React 19 + TypeScript
- Autenticación (login/registro) conectada a auth-service
- Listado de eventos desde event-service
- Vista "Mis Eventos" con inscripciones del usuario
- Perfil de usuario conectado a user-service
- Proxy configurado: /api → Gateway:8080

## Pruebas realizadas
- [ ] inscrip-service compila sin errores
- [ ] frontend-v2 compila sin errores (npm run build)
- [ ] Frontend carga en http://localhost:5173
- [ ] Login funciona
- [ ] Se pueden ver eventos
- [ ] Se puede inscribir a un evento
```

3. Clic "Create Pull Request"

Por CLI:

```
gh pr create \
  --base develop \
  --head feature/eliana-inscrip-frontend \
  --title "feat(inscrip+frontend): inscrip-service y frontend-v2 completos" \
  --body "inscrip-service: CRUD de inscripciones + frontend-v2: React SPA con todas las páginas funcionales"
```

 **ACTUALIZAR TU RAMA CON DEVELOP**

```
git checkout develop
git pull origin develop
git checkout feature/eliana-inscrip-frontend
git merge develop
# No debería haber conflictos porque tus carpetas son únicas
git push origin feature/eliana-inscrip-frontend
```

PROBLEMAS COMUNES Y SOLUCIONES

frontend-v2: "npm run build" falla con errores TypeScript

```
cd frontend-v2

# Ver errores específicos
npm run build 2>&1

# Si es un error de tipos
npm run lint

# Solución común: revisar importaciones en el archivo con error
```

frontend-v2: node_modules apareció en git status

```
# Verificar .gitignore
cat .gitignore | grep node_modules

# Si no está, agrega a .gitignore (raíz o frontend-v2/.gitignore):
# node_modules/

# Eliminar del tracking de git (sin borrar localmente)
git rm -r --cached frontend-v2/node_modules/
git add frontend-v2/node_modules/ # Esto NO los agrega si están en .gitignore
```

inscrip-service: error de compilación JPA

```
cd inscrip-service
./mvnw package -DskipTests 2>&1 | grep ERROR
# Revisar los errores específicos de entidades/relaciones JPA
```

"Port 8084 already in use"

```
# Windows
netstat -ano | findstr :8084
taskkill /PID [PID] /F
```

El frontend no conecta con el backend (CORS)

```
# Verificar que el gateway está corriendo en puerto 8080
curl http://localhost:8080/health

# Verificar la URL base en api.ts
# En desarrollo debe ser /api (proxy de Vite)
# El proxy de Vite está configurado en vite.config.ts
cat frontend-v2/vite.config.ts
```



CHECKLIST PERSONAL — ELIANA

Preparación

- ☐ Esperé confirmación de Rachel (Fases 1-4 mergeadas)
- ☐ `git checkout develop && git pull origin develop` ejecutado
- ☐ Veo commits de todos en `git log`
- ☐ Creé rama `feature/eliana-inscrip-frontend`

Verificación de compilación

- ☐ `inscrip-service` compila: `./mvnw package -DskipTests`
- ☐ `frontend-v2` compila: `npm run build`

Antes del commit

- ☐ Solo `inscrip-service/` y `frontend-v2/` en staging
- ☐ `node_modules/` NO está en staging
- ☐ Mensaje de commit descriptivo

PR

- ☐ Push exitoso a GitHub
- ☐ Verificado en GitHub que solo aparecen mis cambios
- ☐ **Base:** `develop` ✓
- ☐ PR creado y Rachel notificada
- ☐ Informé a Rachel que este es el último PR (puede proceder con la integración final)



INFORMACIÓN TÉCNICA

inscrip-service:

- Puerto: `8084`
- Base de datos: `inscrip_db` (Railway MySQL)
- Ruta en Gateway: `/inscriptions/**` → `http://localhost:8084`

Endpoints de inscrip-service:

GET	<code>/inscriptions</code>	→ Listar inscripciones
GET	<code>/inscriptions/{id}</code>	→ Obtener inscripción
GET	<code>/inscriptions/user/{userId}</code>	→ Inscripciones de un usuario
POST	<code>/inscriptions</code>	→ Crear inscripción
DELETE	<code>/inscriptions/{id}</code>	→ Cancelar inscripción
GET	<code>/inscriptions/health</code>	→ Health check

frontend-v2:

- Puerto de desarrollo: 5173
- Framework: React 19 + TypeScript + Vite
- Producción: Railway (<https://frontend-production-5e8b.up.railway.app>)

Flujo del frontend:

```
Usuario → localhost:5173 (Vite dev)
      ↓
    /api/* (proxy Vite)
      ↓
  localhost:8080 (Gateway)
      ↓
  Servicio correspondiente
```

Guía preparada para: Eliana | Proyecto: Eventos-Microservicios | 2026-03-13